

A Study of HAWK: Algorithms, Parameters, and Cryptanalysis

Keshav Rajput

CS-UY 3943 Post-Quantum Cryptography, Spring 2026

Professor: Dr. Delaram Kahrobaei

June 08, 2026

Abstract

HAWK is a lattice-based post-quantum digital signature scheme submitted to NIST’s call for additional post-quantum signature standards. The scheme draws its security from the Lattice Isomorphism Problem (LIP) and two closely related computational problems: the one-more (approximate) Shortest Vector Problem (omSVP) and the search module Lattice Isomorphism Problem (smLIP). HAWK employs a hash-and-sign design paradigm over power-of-two cyclotomic rings and operates on a secret unimodular matrix basis $B \in GL_2(\mathcal{R}_n)$, where the public key is the Gram matrix $Q = B^*B$. Two main parameter sets are provided—HAWK-512 and HAWK-1024—targeting NIST security levels I and V, respectively, along with a challenge set HAWK-256. The scheme is notably lightweight: HAWK-512 requires no more than 14 KiB of RAM for any operation, is entirely free of floating-point arithmetic, and achieves isochronous execution with respect to secret data. Signature generation samples from two fixed discrete Gaussian distributions over integer cosets, while key generation solves an NTRU equation over the ring $\mathcal{R}_n = \mathbb{Z}[X]/(X^n + 1)$. HAWK’s verification procedure reconstructs internal signing variables from the public key and signature and checks a norm bound under the Q -norm. This report presents a study of HAWK’s algorithms, parameter selections, formal security intuitions, and cryptanalysis, with emphasis on its suitability for constrained embedded environments in the post-quantum era.

Contents

1	Introduction	2
2	Background and Preliminaries	3
2.1	Cyclotomic Rings	3
2.2	Distributions	3
2.3	Rényi Divergence	3
2.4	The NTRU Equation	4
2.5	Hardness Assumptions	4
3	Description of the Scheme	4
4	Algorithms of the Scheme	5
4.1	Key Generation	5

4.2	Signature Generation	6
4.3	Signature Verification	6
5	Cryptanalysis	7
5.1	Attack Models	7
5.2	Forgery and omSVP	7
5.3	Security Estimates via BKZ	7
5.4	Key Recovery and smLIP	8
5.5	Side-Channel Considerations	8
6	Parameters and Efficiency	8
7	Running Time / Implementation Notes	9
8	Conclusion	10
9	Current Relevance and Practical Status	11
9.1	Standardisation Status	11
9.2	Theoretical Security: Strengths and Concerns	11
9.3	Practical Security: Deployment Readiness	11

1 Introduction

The imminent threat of large-scale quantum computers to classic public-key cryptography has made the standardisation of post-quantum cryptographic schemes a matter of urgency. Shor’s algorithm can efficiently solve both the integer factorisation and discrete logarithm problems, undermining the security of RSA, DSA, ECDSA, and related schemes [2]. The U.S. National Institute of Standards and Technology (NIST) has run a multi-round competition to identify and standardise quantum-resistant algorithms; the first batch of signature standards was announced in 2024, including CRYSTALS-Dilithium, FALCON, and SPHINCS+ [3]. Seeking additional diversity, NIST opened a second call for post-quantum signature schemes, to which HAWK was submitted as a Round 2 candidate [1].

HAWK is a hash-and-sign lattice-based signature scheme whose hardness assumptions differ meaningfully from those of the already-standardised FALCON [1]. While FALCON relies on the hardness of the Short Integer Solution (SIS) problem over NTRU lattices, HAWK’s forgery resistance is reduced to the one-more approximate Shortest Vector Problem (omSVP), and secret-key recovery is related to the search module Lattice Isomorphism Problem (smLIP). These are relatively new cryptographic assumptions, and HAWK therefore represents a diversification of the algebraic footprint of the post-quantum signature portfolio [1].

The design of HAWK prioritises three goals: speed, compactness, and suitability for constrained hardware. Signature generation and verification are fast on all tested platforms including ARM Cortex-M4 embedded processors, and the scheme requires no floating-point unit, making it viable for a wide class of microcontrollers [1]. Key and signature sizes are compact: HAWK-512 public keys are 1024 bytes and signatures are 555 bytes, while HAWK-1024 public keys are 2440 bytes and signatures are 1221 bytes.

This report provides a structured study of HAWK version 1.1 as described in the official specification document submitted to NIST Round 2 [1]. We present the mathematical background (Section 2), scheme description (Section 3), detailed algorithms (Section 4), cryptanalysis (Section 5), parameters and efficiency (Section 6), implementation notes (Section 7), and conclusion (Section 8).

2 Background and Preliminaries

2.1 Cyclotomic Rings

HAWK operates over the power-of-two cyclotomic number field $K_n = \mathbb{Q}[X]/(X^n + 1)$ and its ring of integers $\mathcal{R}_n = \mathbb{Z}[X]/(X^n + 1)$, where $n \in \{256, 512, 1024\}$ [1]. Polynomials in K_n are identified with their coefficient vectors via the bijection $\text{vec} : K_n \rightarrow \mathbb{Q}^n$, $f \mapsto (f[0], \dots, f[n-1])$. Multiplication by $f \in K_n$ acts as a linear map on K_n , and the corresponding matrix is the negacyclic matrix $\text{rot}(f) \in \mathbb{Q}^{n \times n}$. The Hermitian adjoint of f is defined as

$$f^* = f[0] - f[n-1]X - \dots - f[1]X^{n-1},$$

and for a matrix $B \in K_n^{r \times r}$, B^* is obtained by applying the adjoint entry-wise to the transpose B^t . These structures are central to defining the public key and the Q -norm used in verification.

2.2 Distributions

Key generation in HAWK samples polynomial coefficients i.i.d. from the centred binomial distribution $\text{Bin}(\eta)$, where η is a security-level parameter. If $X \sim \text{Bin}(\eta)$ then

$$\Pr[X = x] = \frac{1}{2^{2\eta}} \binom{2\eta}{x + \eta} \quad \text{for } x \in \{-\eta, \dots, \eta\},$$

with $\mathbb{E}[X] = 0$ and $\text{Var}[X] = \eta/2$ [1]. Sampling is achieved simply by XOR-ing pairs of uniform bits.

Signature generation uses the discrete Gaussian distribution $D_{2\mathbb{Z}+c, \sigma}$ on the coset $2\mathbb{Z}+c$ with parameter $\sigma > 0$, defined via

$$\Pr[X = x] = \frac{\rho_\sigma(x)}{\sum_{z \in 2\mathbb{Z}+c} \rho_\sigma(z)}, \quad \rho_\sigma(x) = \exp\left(-\frac{x^2}{2\sigma^2}\right).$$

Only $c \in \{0, 1\}$ is used in HAWK, and sampling is realised via two fixed precomputed tables, avoiding any key-dependent or data-dependent timing [1].

2.3 Rényi Divergence

The security analysis of HAWK makes extensive use of the Rényi divergence of order $a \in (1, \infty)$ between distributions P and Q [1]:

$$R_a(P||Q) = \left(\sum_{x \in \text{Supp}(P)} \frac{P(x)^a}{Q(x)^{a-1}} \right)^{1/(a-1)}.$$

Key properties include the data processing inequality, multiplicativity over product distributions, and a probability preservation bound that allows the security of an approximated distribution to be related to that of the ideal distribution [1]. These tools are used to justify the use of finite-precision discrete Gaussian samplers in lieu of ideal ones.

2.4 The NTRU Equation

The HAWK key generation requires finding $F, G \in \mathcal{R}_n$ satisfying the NTRU equation

$$fG - gF = 1 \pmod{X^n + 1},$$

given small polynomials $f, g \in \mathcal{R}_n$ [1]. The algorithm NTRUSOLVE computes this decomposition when it exists; failure triggers a restart. This is the dominant computational cost in key generation and is analogous to FALCON’s key generation procedure.

2.5 Hardness Assumptions

omSVP. The one-more approximate Shortest Vector Problem requires an adversary that is given polynomially many approximate shortest vectors in a lattice to find one more such vector it has not been given. Breaking the SUF-CMA security of HAWK reduces to solving omSVP in the quantum random oracle model [1].

smLIP. The search module Lattice Isomorphism Problem requires, given a Gram matrix $Q = B^*B$, finding the secret basis B . Recovering the HAWK private key from the public key is exactly an instance of rank-2 smLIP over a CM-field [1].

Practical security from new assumptions. Both omSVP and smLIP are newer than the SIS/LWE assumptions underlying FALCON and Dilithium, which raises a fair question: does novelty hurt security? For omSVP, the intuition is that the “one-more” structure is strictly harder than a single SVP query—any algorithm that broke HAWK could be used to solve BKZ-hard lattice problems, but not vice versa, so HAWK parameters are independently calibrated against BKZ experiments rather than derived purely from the reduction. For smLIP over the CM-field, the key observation is that an attacker must abandon the ring structure and work with the full unstructured lattice $\text{rot}(B) \in \mathbb{Q}^{2n \times 2n}$, a dimension- $2n$ problem. This is why the practical BKZ estimates in Section 5 provide genuine confidence in the parameters even when the theoretical reduction loss is large.

3 Description of the Scheme

HAWK is a hash-and-sign signature scheme built over the ring $\mathcal{R}_n$. The private key is a unimodular basis matrix

$$B = \begin{pmatrix} f & F \\ g & G \end{pmatrix} \in GL_2(\mathcal{R}_n),$$

where f, g have small coefficients and f, g, F, G satisfy $fG - gF = 1 \pmod{X^n + 1}$ [1]. The public key is the Hermitian Gram matrix $Q = B^*B$, which equals

$$Q = \begin{pmatrix} f^*f + g^*g & f^*F + g^*G \\ F^*f + G^*g & F^*F + G^*G \end{pmatrix}.$$

Due to the NTRU relation, $q_{00}q_{11} - q_{01}q_{10} = 1$, so the entire matrix Q is determined by only q_{00} and q_{01} . The private key is stored compactly as: a random seed `kgseed` (from which f, g are regenerated deterministically), the parities $F \bmod 2$ and $G \bmod 2$, and $h_{\text{pub}} = \text{SHAKE256}(Q)$.

The high-level design follows the GPV hash-and-sign framework [4]: to sign message m , the signer hashes m to obtain a binary vector $h \in \mathcal{R}_n^2$, then uses the secret basis B to find a short vector w in the coset $2\mathcal{R}_n^2 + Bh$. The signature encodes a compressed version of $s = (h - w)/2$. Verification recomputes w from the signature and public key, and accepts if $\|w\|_Q^2 \leq 4\sigma_{\text{verify}}^2 \cdot 2n$.

A symmetry-breaking condition **SYM-BREAK** is imposed to prevent trivial forgeries that exploit the sign-flip ambiguity $w \mapsto -w$. A notable design choice is to include only the second component s_1 of the vector $s = (s_0, s_1)$ in the signature. The verifier reconstructs s_0 deterministically using the public key and the relation

$$s_0 = \frac{h_0}{2} + \frac{q_{01}}{q_{00}} \left(\frac{h_1}{2} - s_1 \right),$$

which reduces signature size substantially. The message is processed via two sequential SHAKE256 calls, first hashing m with h_{pub} to form M , then hashing M with a uniform salt to form h . This two-level hashing structure was originally designed to accommodate the BUFF transform; subsequent work showed HAWK natively satisfies the BUFF security properties [1].

Object encoding (implementor note). While the full encoding and decoding machinery (Algorithms 4–11 in the specification [1]) is beyond the scope of this report, the key design choice is that signatures are compressed using a Golomb-Rice-style scheme parameterised by `lows1`, `highs1`, and `highs0` values tuned per parameter set. The public key encodes q_{00} and q_{01} with separate low/high bit-length parameters (e.g., `low00=5`, `high00=9` for HAWK-512). This encoding is what produces the compact byte sizes in Table 2; the encoding step can itself fail and trigger a restart, though with negligible probability.

4 Algorithms of the Scheme

4.1 Key Generation

Key generation first samples a random seed and derives $f, g \in \mathcal{R}_n$ with coefficients from the centred binomial distribution [1]. Several conditions on f, g are checked (including that they have odd parity and that $\|(f, g)\|$ is not too small) to ensure the subsequent NTRUSOLVE step can proceed efficiently. The NTRUSOLVE algorithm finds $F, G \in \mathcal{R}_n$ satisfying the NTRU equation; a reduction step is applied to keep F, G small, using methods from [6, 7]. The most computationally expensive step is this NTRU basis completion and reduction, which takes on the order of 8.43×10^6 clock cycles for HAWK-512 on an x86 Coffee Lake CPU [1].

Algorithm 1 HAWKKEYGEN(n, η)

```

1: Sample kgseed  $\leftarrow$  Rnd(kgseedlen bits)
2: Derive  $f, g \in \mathcal{R}_n$  from kgseed deterministically, each i.i.d.  $\sim \text{Bin}(\eta)$ 
3: if  $\neg \text{F-G-CONDITIONS}(f, g)$  then restart
4:  $r \leftarrow \text{NTRUSOLVE}(f, g)$ 
5: if  $r = \perp$  then restart
6:  $(F, G) \leftarrow r$ ; reduce  $(F, G)$  to make entries small
7:  $B \leftarrow \begin{pmatrix} f & F \\ g & G \end{pmatrix}$ ;  $Q \leftarrow B^* B$ 
8: if  $\neg \text{KGEN-ENCODING}(Q, B)$  then restart
9:  $h_{\text{pub}} \leftarrow \text{SHAKE256}(Q)$ 
10:  $sk \leftarrow (\text{kgseed}, F \bmod 2, G \bmod 2, h_{\text{pub}})$ ;  $pk \leftarrow Q$ 
11 return  $(pk, sk)$ 

```

Figure 1: HAWK key generation algorithm.

4.2 Signature Generation

Algorithm 2 HAWKSIGN(sk, m)

```

1: Decode  $sk$ ; recompute  $f, g$  from kgseed
2:  $M \leftarrow \text{SHAKE256}(m \| h_{\text{pub}})$ 
3: salt  $\leftarrow$  Rnd(saltlen bits)
4:  $h \leftarrow \text{SHAKE256}(M \| \text{salt})$  ( $2n$  bits, interpreted as  $(h_0, h_1) \in \mathcal{R}_n^2$ )
5:  $t \leftarrow B \cdot h \bmod 2$ 
6:  $x \leftarrow D_{2\mathbb{Z}^{2n}+t, 2\sigma_{\text{sign}}}$ 
7: if  $\|x\|^2 > 4\sigma_{\text{verify}}^2 \cdot 2n$  then restart
8:  $w \leftarrow B^{-1}x$ 
9: if  $\neg \text{SYM-BREAK}(w)$  then  $w \leftarrow -w$ 
10:  $s_1 \leftarrow \text{COMPRESS}((h_1 - w_1)/2)$ 
11 if  $\neg \text{SIG-ENCODING}(\text{salt}, s_1)$  then restart
12 return  $\text{sig} \leftarrow (\text{salt}, s_1)$ 

```

Figure 2: HAWK signature generation algorithm.

Signing computes two SHAKE256 hashes: $M = \text{SHAKE256}(m \| h_{\text{pub}})$ and $h = \text{SHAKE256}(M \| \text{salt})$. The target coset is $2\mathbb{Z}^{2n} + t$ with $t = Bh \bmod 2$. A discrete Gaussian vector x is sampled over this coset; its inverse-basis image $w = B^{-1}x$ lies in $2\mathcal{R}_n^2 + h$ and is short under the Q -norm [1]. After applying SYM-BREAK, only the second component $s_1 = (h_1 - w_1)/2$ is encoded and included in the signature, reducing signature length. The restart probability is approximately $1/200,000$ for HAWK-512 and $1/400,000$ for HAWK-1024 [1], giving essentially deterministic worst-case running time.

4.3 Signature Verification

Verification is straightforward and requires only the public key Q and the signature (salt, s_1) [1]. The verifier recomputes h as in signing, reconstructs $w_1 = h_1 - 2s_1$, checks the symmetry-breaking condition, then rebuilds s_0 using integer fixed-point arithmetic, and computes $w_0 = h_0 - 2s_0$. Finally, the Q -norm of w is compared against the verification threshold. The entire verification procedure for HAWK-512 takes

Algorithm 3 HAWKVERIFY(pk, m, sig)

```

1: Decode  $pk = Q$ ; decode  $sig = (\text{salt}, s_1)$ 
2:  $h_{\text{pub}} \leftarrow \text{SHAKE256}(Q)$ ;  $M \leftarrow \text{SHAKE256}(m \| h_{\text{pub}})$ ;  $h \leftarrow \text{SHAKE256}(M \| \text{salt})$ 
3:  $w_1 \leftarrow h_1 - 2s_1$ 
4: if  $\neg \text{SYM-BREAK}(w_1)$  then return REJECT
5:  $s_0 \leftarrow \frac{h_0}{2} + \frac{q_{01}}{q_{00}} \left( \frac{h_1}{2} - s_1 \right)$  (32-bit fixed-point arithmetic)
6:  $w_0 \leftarrow h_0 - 2s_0$ ;  $w \leftarrow (w_0, w_1)$ 
7: if  $\|w\|_Q^2 \leq 4\sigma_{\text{verify}}^2 \cdot 2n$  then return ACCEPT else return REJECT

```

Figure 3: HAWK signature verification algorithm.

approximately 1.48×10^5 cycles on x86 Coffee Lake [1]. Since all computations are over public data, verification is free of timing side channels.

5 Cryptanalysis

5.1 Attack Models

HAWK targets strong unforgeability under chosen-message attacks (SUF-CMA), meaning an adversary with access to a signing oracle should not be able to produce any valid (message, signature) pair for a message it has not queried, nor produce a new signature for a previously queried message [1]. Two distinct attack targets exist: (1) forging a signature without knowledge of the private key, and (2) recovering the private key from the public key.

5.2 Forgery and omSVP

A strong forgery of HAWK requires the adversary to find, for a uniformly random binary vector $h \in \mathcal{R}_n^2$, a vector $w \in 2\mathcal{R}_n^2 + h$ satisfying $\|w\|_Q \leq 2\sigma_{\text{verify}}\sqrt{2n}$ [1]. The SUF-CMA security of HAWK is formally reduced to omSVP in the quantum random oracle model (qROM) [1]. The analysis uses Rényi divergence tools to handle the slight mismatch between the ideal and implemented distributions. An important subtlety is that the omSVP parametrisation implied by the reduction is one that *can* in principle be efficiently solved; the HAWK authors therefore rely on independent BKZ experiments to justify parameter choices, not the reduction alone.

5.3 Security Estimates via BKZ

The primary experimental cryptanalysis tool is lattice basis reduction via the BKZ algorithm. For HAWK-512, the forgery lattice has dimension $2n = 1024$, and security is estimated using the core-SVP methodology [1]. Table 1 gives compressed best-case and worst-case security estimates across both attack types and all parameter sets.

Even in the worst case, HAWK-512 and HAWK-1024 comfortably exceed their target NIST security levels I and V, respectively. The gap between best and worst estimates reflects uncertainty in concrete BKZ costs; HAWK-256 is an intentional challenge set rather than a deployment target.

Table 1: Core-SVP security estimates for HAWK [1]. Best = optimistic defender model; Worst = accounting for known BKZ speedups.

Parameter set	Forgery (omSVP)		Key Recovery (smLIP)	
	Best (bits)	Worst (bits)	Best (bits)	Worst (bits)
HAWK-256	≈ 64	≈ 55	≈ 64	≈ 55
HAWK-512	≈ 128	≈ 110	≈ 128	≈ 110
HAWK-1024	≈ 256	≈ 220	≈ 256	≈ 220

5.4 Key Recovery and smLIP

Private key recovery from the public key $Q = B^*B$ is equivalent to rank-2 smLIP over a CM-field [1]. Recent theoretical advances (2024–2025) have clarified this significantly:

- [9]: A classical polynomial-time algorithm for rank-2 smLIP over *totally real* fields was given. HAWK operates over the CM-field (not totally real), which is not covered without solving a quaternionic principal ideal problem [1].
- [10]: Extending the above, breaking smLIP over CM-fields requires solving a principal ideal problem over a quaternionic extension, currently not known to be efficiently solvable [1].
- [11]: A symplectic automorphism approach reduces to a commutative problem, but only efficiently over totally real fields [1].
- [12]: Extended classical attacks to number fields with at least one real embedding, but this still does not cover CM-fields [1].

The conclusion is that rank-2 smLIP over a CM-field remains open to efficient attack. The HAWK team monitors this research carefully and updated the specification in version 1.1 accordingly.

5.5 Side-Channel Considerations

HAWK achieves isochronous execution for all secret-data-dependent procedures by design [1]. The Gaussian sampling tables are fixed and independent of the key or message. Recent work [8] experimentally demonstrated power-analysis attacks against the Gaussian sampler in the reference implementation and proposed low-overhead countermeasures; the authors report these thwart the demonstrated attacks, though provable side-channel resistance has not yet been established. Masking the Gaussian sampler remains an open research problem [1].

6 Parameters and Efficiency

Table 2 summarises the three HAWK parameter sets as specified in the official specification document [1].

Table 3 summarises the performance benchmarks reported in the specification [1].

Table 4 shows the RAM usage reported for HAWK operations [1].

Table 2: Reported parameters for HAWK [1].

Parameter	HAWK-256	HAWK-512	HAWK-1024
Targeted security	Challenge (64-bit)	NIST-I (128-bit)	NIST-V (256-bit)
Degree n	256	512	1024
Binomial param. η	2	4	8
σ_{sign}	1.010	1.278	1.299
σ_{verify}	1.042	1.425	1.571
Salt length (bits)	112	192	320
Private key (bytes)	96	184	360
Public key (bytes)	450	1024	2440
Signature (bytes)	249	555	1221
Max. transcript q_s	2^{32}	2^{64}	2^{64}

Table 3: Speed of HAWK in clock cycles [1]. Bracketed values use less RAM.

Operation	HAWK-512 (x86)	HAWK-1024 (x86)
Keypair generation	8.43×10^6	4.37×10^7
Signature generation	8.54×10^4 (4.37×10^4)	1.81×10^5 (8.54×10^4)
Signature verification	1.48×10^5 (1.24×10^5)	3.03×10^5 (2.55×10^5)
Operation	HAWK-512 (ARM M4)	HAWK-1024 (ARM M4)
Keypair generation	5.23×10^7	2.26×10^8
Signature generation	2.80×10^6 (1.16×10^6)	—
Signature verification	1.42×10^6 (1.23×10^6)	3.01×10^6 (2.61×10^6)

The most compact configuration, HAWK-512, uses at most 14 KiB of RAM for any operation, well within the 16 KiB SRAM found on common ARM Cortex-M0(+) microcontrollers such as the NXP LPC800 series, STM32F0, and Infineon XMC1000 [1]. If keys are generated externally and hard-coded, signing and verification require only 6 KiB and 11 KiB respectively for HAWK-1024 [1]. The absence of floating-point arithmetic is a significant advantage over FALCON, which requires a double-precision FPU.

7 Running Time / Implementation Notes

The specification provides two reference implementations: a pedagogically clear Python 3 implementation (with one explicitly noted deviation from the spec for clarity), and an optimised C implementation that is lightweight, fast, isochronous, and floating-point free [1]. The C implementation uses fixed-point 32-bit integer arithmetic throughout, including for the s_0 reconstruction step in verification.

Key generation is the most expensive phase, dominated by the NTRUSOLVE step. On x86 with AVX2 instructions, HAWK-512 key generation takes approximately 8.43×10^6 cycles and HAWK-1024 takes 4.37×10^7 cycles [1]. The optimised key generation combines basis completion and reduction using published methods [6, 7].

Signing is fast in practice. The restart probability is approximately 1/200,000 for HAWK-512 and

Table 4: RAM usage of HAWK in bytes [1]. Bracketed values correspond to speed-optimised variants.

Operation	HAWK-512	HAWK-1024
Keypair generation	14,336	27,648
Signature generation	4,096 (5,272)	7,168 (9,512)
Signature verification	6,144 (8,768)	11,264 (16,512)

1/400,000 for HAWK-1024, giving essentially deterministic throughput [1]. The primary cost of signing on embedded platforms is the discrete Gaussian sampling, which is implemented via two fixed lookup tables requiring no floating-point operations. The isochronous property means both the table-lookup and all subsequent operations run in time independent of the sampled value.

Verification is the most lightweight operation and makes no secret data access after key decoding. The Q -norm bound check uses only integer arithmetic and the public matrix Q . The fixed-point reconstruction of s_0 is performed with 32-bit integers; the scheme parameters ensure negligible failure probability [1]. Future work proposed by the HAWK team includes efficient masking of the Gaussian sampler for side-channel-protected deployments, which remains an open problem as of the Round 2 submission.

Connection to security. These implementation choices are not merely engineering conveniences—they are load-bearing components of the security argument. The fixed, key-independent Gaussian lookup tables are precisely what allows the Rényi divergence analysis in Section 2.3 to go through: if the tables were key-dependent or message-dependent, the divergence bound would fail, and the reduction to omSVP would break. Similarly, the 32-bit fixed-point reconstruction of s_0 in verification is possible only because the parameters $(\sigma_{\text{sign}}, \sigma_{\text{verify}}, n)$ are co-designed to keep the reconstruction error negligible—a tighter σ_{verify} would improve forgery hardness on paper but would cause the integer approximation to fail in practice. This co-design between parameter selection and integer arithmetic is a concrete example of practical security considerations feeding back into the theoretical design, and it illustrates why a scheme that looks correct on paper must be evaluated as a whole system.

8 Conclusion

HAWK is a compelling post-quantum signature candidate that occupies a distinct niche compared to NIST’s already-standardised schemes. Its use of novel hardness assumptions—omSVP and smLIP—over power-of-two cyclotomic CM-fields provides cryptographic diversity relative to the SIS/LWE-based CRYSTALS-Dilithium and FALCON [1]. The scheme is compact, fast, floating-point free, and memory efficient, making it particularly attractive for IoT and embedded applications with severe hardware constraints.

The primary open question is the long-term hardness of smLIP over CM-fields. Recent results have established polynomial-time classical attacks on smLIP over totally real fields [9], and there is active progress on the CM-field case via quaternionic principal ideal problems and symplectic automorphisms [10, 11]. The HAWK team monitors this research carefully and has updated the specification accordingly in version 1.1. The omSVP assumption is also new: because the reduction does not directly justify the parameter sizes, practical security rests on the empirical BKZ hardness estimates reviewed in Section 5—a methodologi-

cal trade-off the authors acknowledge explicitly. HAWK currently participates in NIST’s additional post-quantum signature process (Round 2), and the challenge parameter set HAWK-256 provides an accessible cryptanalytic target whose resolution would constitute significant scientific progress.

9 Current Relevance and Practical Status

9.1 Standardisation Status

As of June 2026, HAWK is an active Round 3 candidate in NIST’s Additional Digital Signatures for the PQC Standardization Process [13]. In May 2026, NIST announced that nine algorithms—including HAWK—advanced from 14 second-round candidates to the third round, after roughly 18 months of evaluation [13]. HAWK is notably the *only lattice-based scheme* in the competition; all other advanced lattice-based signatures (FALCON/FN-DSA, ML-DSA) have already been standardised from the main PQC process. Submission teams may submit revised specifications by August 14, 2026, and the third round is expected to last approximately two years before further standardisation decisions are made [13].

9.2 Theoretical Security: Strengths and Concerns

Theoretically, HAWK occupies a healthy but uncertain position.

Strengths. HAWK provides cryptographic diversity: its omSVP and smLIP assumptions are independent of the SIS/LWE problems underlying ML-DSA and FN-DSA, so a mathematical breakthrough against structured lattices would not automatically break HAWK. The scheme provably achieves SUF-CMA security in the qROM [1], a stronger security notion than EUF-CMA, which is important for avoiding replay and double-spend attacks in real protocols. Recent (2024–2025) analysis confirmed that the CM-field case of smLIP—the exact case HAWK uses—remains open to efficient attack despite polynomial-time algorithms being found for the totally real subfield [1]. NIST’s second-round report explicitly noted that recent cryptanalysis did not produce practical attacks against HAWK [13].

Concerns. NIST simultaneously encouraged additional analysis of HAWK’s underlying security assumptions when advancing it to Round 3 [13]. The core tension is that both omSVP and smLIP are newer assumptions with less accumulated cryptanalytic scrutiny than SIS/LWE, which have been studied since the 1990s. As noted in Section 5, the omSVP parametrisation implied by the formal reduction can in principle be efficiently solved, so the practical security case rests on BKZ experiments rather than a tight reduction—a methodological gap. The active research progress on smLIP (four major papers in 2024–2025 alone) reflects genuine uncertainty about the long-term hardness of the CM-field case. If an efficient quantum algorithm for quaternionic principal ideal problems were found, HAWK’s key recovery assumption would fall.

9.3 Practical Security: Deployment Readiness

Practically, HAWK is not yet appropriate for production deployment, but for well-understood reasons:

- **Not yet standardised.** HAWK has no FIPS designation. For regulated environments (U.S. federal, finance, healthcare), NIST mandates the use of standardised algorithms. Organisations should deploy ML-DSA or FN-DSA now and treat HAWK as a candidate for future cryptographic agility [3].
- **Hardware suitability is excellent.** HAWK-512 requires only 14 KiB RAM, is floating-point free, and runs in under 0.1 ms on an average desktop [1]. This makes it uniquely attractive for IoT, embedded firmware signing, and microcontrollers that cannot support FALCON’s FPU requirement. If HAWK is standardised, it would fill a niche currently unserved by the existing FIPS standards.
- **Implementation maturity is reasonable.** Both a reference Python implementation and an AVX2-optimised C implementation are publicly available at <https://hawk-sign.info> [14]. No side-channel findings during the second round were severe enough to disqualify HAWK, though the Gaussian sampler masking problem remains open [13].
- **Quantum resistance is well-argued.** HAWK’s security does not rely on any problem solvable by Shor’s algorithm. The BKZ-based estimates in Table 1 are based on classical lattice reduction, and no quantum speedup analogous to Shor’s is known for BKZ on these lattices.

In summary, HAWK is a well-designed, theoretically sound candidate whose main vulnerability is the relative youth of its hardness assumptions. It is practically useful for constrained hardware scenarios and poised for standardisation if the CM-field smLIP assumption withstands ongoing scrutiny over the next two years of Round 3 review.

References

- [1] J. W. Bos, O. Bronchain, L. Ducas, S. Fehr, Y.-H. Huang, T. Pornin, E. W. Postlethwaite, T. Prest, L. N. Pulles, and W. van Woerden, *HAWK v1.1: Round-2 Specification Document*, NIST PQC Standardization, February 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/hawk-spec-round2-web.pdf>
- [2] P. W. Shor, “Algorithms for quantum computation: Discrete logarithms and factoring,” in *Proc. 35th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, 1994, pp. 124–134.
- [3] National Institute of Standards and Technology, *Post-Quantum Cryptography Standardization*, <https://csrc.nist.gov/projects/post-quantum-cryptography>, 2024.
- [4] C. Gentry, C. Peikert, and V. Vaikuntanathan, “Trapdoors for hard lattices and new cryptographic constructions,” in *Proc. 40th Annual ACM Symposium on Theory of Computing (STOC)*, 2008, pp. 197–206.
- [5] L. Ducas and W. van Woerden, “HAWK: Module LIP makes lattice signatures faster, smaller and more standard,” in *Proc. ASIACRYPT 2022*, LNCS, vol. 13794, Springer, 2022, pp. 65–98.
- [6] T. Prest and T. Pornin, “Efficient NTRU lattice-based signature scheme,” IACR ePrint, 2019.
- [7] T. Pornin, “NTRUSolver and associated code,” 2023. <https://hawk-sign.info>

- [8] K. Gärtner and A. Russon, “Power analysis attacks against HAWK’s Gaussian sampler and counter-measures,” IACR ePrint, 2024.
- [9] S. D. Miller, T. Postlethwaite, T. Prest, and W. van Woerden, “Classical polynomial time algorithms for rank two smLIP over totally real fields,” IACR ePrint, 2024.
- [10] H. Cohn, S. Fehr, and A. Menezes, “Module lattice isomorphism and principal ideal problems over CM-fields,” IACR ePrint, 2024.
- [11] C. Lee, M. Joseph, T. Postlethwaite, and W. van Woerden, “Symplectic automorphisms and smLIP,” IACR ePrint, 2024.
- [12] A. P. van Woerden et al., “Extended classical polynomial-time attack to number fields with real embeddings,” IACR ePrint, 2025.
- [13] D. Moody et al., *Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process*, NIST IR 8610, May 2026. <https://doi.org/10.6028/NIST.IR.8610>
- [14] HAWK Signatures Project, <https://hawk-sign.info>, accessed June 2026.